

Reviewer Pack — Newton-Inequality Defect of SAT Generating Polynomial Bounds...

Entry 4907d4d1fa3e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 08:18:29 UTC

Newton-Inequality Defect of SAT Generating Polynomial Bounds DPLL Depth

Entry ID: 4907d4d1fa3e **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 08:18:29 UTC

1. Conjecture

Field A (mathematical branch): Pólya-Schur theory of real-rooted polynomials and Newton's inequalities (multiplier sequences, log-concavity defects of integer coefficient polynomials) **Field B** (complexity object): Average-case DPLL search-tree depth on random 3-CNF and its direct-product worst-case lift

Statement:

For a 3-CNF F on n variables, define the solution generating polynomial $\Phi_F(z) = \sum_{k=0}^n a_k(F) z^k$ where $a_k(F) = \#\{x \in \{0,1\}^n : F(x)=1 \text{ and } \text{weight}(x)=k\}$, and let the Newton-inequality defect be $\delta(F) = \max_{1 \leq k \leq n-1} \max(0, \frac{a_{k-1} a_{k+1}}{a_k^2} - \frac{(k-1)(n-k+1)}{(k(n-k+1))})$ with the convention $0/0 = 0$. Conjecture: there exist constants $c_1, c_2 > 0$ (independent of n) such that for random satisfiable 3-CNF F at clause density α in $[3.6, 4.0]$, the expected average-case DPLL leaf count satisfies $E[\log_2 \text{leaves_DPLL}(F)] \geq c_1 * n * \log_2(1 + \delta(F)) - c_2 * \sqrt{n}$, and the k -fold disjoint-variable direct product $F^{\{(k)\}}$ satisfies $\text{leaves_DPLL}(F^{\{(k)\}}) \geq (1 + \delta(F))^{c_1 * k * n}$ for every static branching order. The conjecture is falsified by exhibiting a single F (with $n \leq 20$) whose measured leaf count is below $c_1 * n * \log_2(1 + \delta(F)) - c_2 * \sqrt{n}$ for $c_1 = 0.25, c_2 = 4$.

Rationale (proposer's reasoning):

Pólya-Schur theory says that polynomials with only real (negative) roots satisfy Newton's inequalities exactly; the defect δ quantifies how far Φ_F is from being a multiplier-stable polynomial. Real-rooted Φ_F corresponds to clusters of solutions arranged like an independent-coordinate product distribution, which DPLL prunes efficiently; conversely, large δ signals pinned correlations between Hamming-weight strata of the solution set, which force DPLL backtracks. Direct product then transports this average-case structural obstruction to a worst-case depth lower bound in the spirit of Impagliazzo-Wigderson hardness amplification.

Taxonomy category: AVG_T0_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a6f6fb85401d4caf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 4 sizes $n \in \{14, 16, 18, 20\} \times 3$ densities $\alpha \in \{3.6, 3.8, 4.0\} \times 30$ satisfiable 3-CNF seeds (360 instances), compute $s_i = \log_2 L(F_i) - 0.25 \cdot n \cdot \log_2(1 + \delta(F_i)) + 4 \cdot \sqrt{n}$ and Pearson r per density bin between $\log_2 L$ and $n \cdot \log_2(1 + \delta)$. SUPPORTED iff fraction $\{s_i \geq 0\} \geq 0.80$ AND median bin-wise $r \geq 0.55$. FALSIFIED iff any seed has $s_i < -3$ OR median bin-wise $r < 0.20$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Newton inequalities log-concave generating polynomial satisfiability DPLL - real-rooted polynomial random 3-SAT solution counting branching complexity - Polya-Schur multiplier sequence SAT search tree lower bound direct product

Top relevant hits considered: - [<http://arxiv.org/abs/1907.08306v1>] A Polynomial Time Algorithm for Log-Concave Maximum Likelihood via Locally Exponential Families - [<http://arxiv.org/abs/2004.07220v2>] Log-Concave Polynomials IV: Approximate Exchange, Tight Mixing Times, and Near-Optimal Sampling of Forests - [<http://arxiv.org/abs/1604.00993v1>] The ‘Core’ of Symmetric Homogeneous Polynomial Inequalities of Degree Four of Three Real Variables - [<http://arxiv.org/abs/1908.05361v2>] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy - [<http://arxiv.org/abs/0803.0018v5>] Another approach to decide on real root existence for univariate Polynomials, and a multivariate extension for 3-SAT - [<http://arxiv.org/abs/1110.1658v2>] Algorithm that Solves 3-SAT in Polynomial Time - [<http://arxiv.org/abs/2312.00421v1>] A Semi-Tensor Product based Circuit Simulation for SAT-sweeping - [<http://arxiv.org/abs/1002.1290v3>] Bounds on Threshold of Regular Random k -SAT

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, alpha):
```

```

clauses = []
for _ in range(int(alpha * n * (n - 1) / 2)):
    clause = [random.randint(0, 1) * 2 - 1 for _ in range(3)]
    random.shuffle(clause)
    clauses.append(clause)
return clauses

def evaluate_formula(formula, assignment):
    return all(all((assignment[var] + lit) % 2 == 0 for var, lit in clause.items()) for clause in clauses)

def dpll(formula, assignment, literals, index=0):
    if not formula:
        return True
    if index == len(literals):
        return False

    literal = literals[index]
    pos_literal = abs(literal) - 1
    new_assignment = assignment[:]
    new_assignment[pos_literal] = (literal > 0)

    if dpll(formula, new_assignment, literals, index + 1):
        return True

    new_assignment[pos_literal] = not new_assignment[pos_literal]
    return dpll(formula, new_assignment, literals, index + 1)

def leaf_count(formula, assignment, literals, index=0):
    if not formula:
        return 1
    if index == len(literals):
        return 0

    literal = literals[index]
    pos_literal = abs(literal) - 1
    new_assignment = assignment[:]
    new_assignment[pos_literal] = (literal > 0)

    count_true = leaf_count(formula, new_assignment, literals, index + 1)
    new_assignment[pos_literal] = not new_assignment[pos_literal]
    count_false = leaf_count(formula, new_assignment, literals, index + 1)

    return count_true + count_false

def newton_inequality_defect(n, clauses):
    a_k = [0] * (n + 1)
    for assignment in product([0, 1], repeat=n):
        if evaluate_formula(clauses, dict(enumerate(assignment))):
            a_k[sum(assignment)] += 1

    delta = 0
    for k in range(1, n):
        a_k_minus_1 = a_k[k-1]
        a_k_plus_1 = a_k[k+1]
        a_k_current = a_k[k]

```

```

        if a_k_current == 0:
            continue

        term = max(0, (a_k_minus_1 * a_k_plus_1) / (a_k_current ** 2) - ((k-1)*(n-k+1)) / (k*(n-k)))
        delta = max(delta, term)

    return delta

def generate_random_satisfiable_3cnf(n, alpha):
    while True:
        clauses = generate_3cnf(n, alpha)
        if any(evaluate_formula(clauses, dict(enumerate([random.randint(0, 1) for _ in range(n)])))):
            return clauses

n_values = [14, 16, 18, 20]
alpha_values = [3.6, 3.8, 4.0]
num_trials = 30
c1, c2 = 0.25, 4

results = []
for n, alpha in product(n_values, alpha_values):
    total_s = 0
    total_r = 0
    count_fails = 0

    for _ in range(num_trials):
        clauses = generate_random_satisfiable_3cnf(n, alpha)
        delta = newton_inequality_defect(n, clauses)

        if delta == 0:
            results.append({"seed": seed, "n": n, "alpha": alpha, "conjecture_holds": False, "count_fails": count_fails})
            continue

        assignment = [random.randint(0, 1) for _ in range(n)]
        leaf_count_value = leaf_count(clauses, assignment, list(range(1, n+1)))

        s = math.log2(leaf_count_value) - c1 * n * math.log2(1 + delta) + c2 * math.sqrt(n)
        r = 0.55 # Placeholder value, will be updated later

        if s < -3:
            count_fails += 1
            results.append({"seed": seed, "n": n, "alpha": alpha, "conjecture_holds": False, "count_fails": count_fails})
            continue

        total_s += s

    if count_fails > 0:
        results.append({"seed": seed, "n": n, "alpha": alpha, "conjecture_holds": False, "count_fails": count_fails})
        continue

    mean_s = total_s / num_trials
    r_values = []

    for _ in range(num_trials):
        clauses = generate_random_satisfiable_3cnf(n, alpha)
        delta = newton_inequality_defect(n, clauses)

```

```

    if delta == 0:
        continue

    assignment = [random.randint(0, 1) for _ in range(n)]
    leaf_count_value = leaf_count(clauses, assignment, list(range(1, n+1)))

    s = math.log2(leaf_count_value) - c1 * n * math.log2(1 + delta) + c2 * math.sqrt(n)
    r_values.append(s / mean_s)

    if len(r_values) > 0:
        r = sum(r_values) / len(r_values)
        results.append({"seed": seed, "n": n, "alpha": alpha, "conjecture_holds": r >= 0.55, '

    return {
        "metric_name": "s_value",
        "metric_value": sum(result["metric_value"] for result in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": all(result["conjecture_holds"] for result in results),
        "counterexample": next((result["counterexample"] for result in results if not result["con]
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60f182d0.py", line 160, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60f182d0.py", line 103, in run_trial
    clauses = generate_random_satisfiable_3cnf(n, alpha)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60f182d0.py", line 88, in generate_rand
    if any(evaluate_formula(clauses, dict(enumerate([random.randint(0, 1) for _ in range(n)]))) for _
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60f182d0.py", line 88, in <genexpr>
    if any(evaluate_formula(clauses, dict(enumerate([random.randint(0, 1) for _ in range(n)]))) for _
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60f182d0.py", line 29, in evaluate_form
    return all(all((assignment[var] + lit) % 2 == 0 for var, lit in clause.items()) for clause in formul
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_60f182d0.py", line 29, in <genexpr>
    return all(all((assignment[var] + lit) % 2 == 0 for var, lit in clause.items()) for clause in formul

```

^^^^^^^^^^^^^^

AttributeError: 'list' object has no attribute 'items'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `AttributeError` in `evaluate_formula` (clauses stored as lists rather than dicts), so no data was produced and neither the support nor falsification criteria can be evaluated. | next: Fix the clause data-structure bug in `evaluate_formula/generate_random_satisfiable_3cnf` (use a consistent representation, e.g., list of (var, lit) tuples) and rerun the full 360-instance pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	27306
2	propose	claude_max	opus	0	0	228558
3	preregistration	claude_max	opus	0	0	9333
4	novelty	claude_max	opus	0	0	5599
5	novelty	claude_max	opus	0	0	9165
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16806
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16948
8	judge	claude_max	opus	0	0	4381

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 318097 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/4907d4d1fa3e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4907d4d1fa3e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4907d4d1fa3e.tar.gz` (if generated)